

班級：資訊三乙 學號：D1210799 姓名：王建葦

一、【實驗目的】：

What was your design? What were the concepts you have used for your design?

本次實驗目的在於熟悉 **RTC (Real-Time Clock)** 與 **PWM (Pulse Width Modulation)** 的應用，並整合 **中斷、ADC、LCD、Keypad、七段顯示器與蜂鳴器**，完成時間顯示與音樂播放系統。

Lab11-1：快轉時鐘 (RTC LCD Clock)

本實驗參考範例程式 `RTC_LCD_Clock`，實作一個可顯示「分針 / 秒針」的時鐘系統：

1. LCD 右半部

- 顯示類比時鐘
- 秒針：每 1 秒移動一小格，數值範圍 00~59
- 分針：每 15 秒移動一大格，數值範圍 00~11，長度比秒針短

2. LCD 左半部

- 顯示目前時間的數字格式 (MM:SS)，例如 01:15
- 分與秒會與右側指針同步更新

3. 操作模式

- 使用 **INT1 (外部中斷)** 切換「運行模式 / 設定模式」
- 進入設定模式時 **PC12 LED 亮起**，時鐘停止
- 只有在設定模式下，Keypad 才能調整時間

4. 時間設定功能

- M 鍵：調整分鐘
- S 鍵：調整秒數
- + / - 鍵：增加或減少
- 若超出合法範圍（如 -3 分或 65 秒），會自動轉換為合法同餘值（如 09 分、05 秒）

本實驗主要練習：

- RTC 時基控制
- 外部中斷 (INT1)
- 狀態切換 (Run / Set Mode)
- LCD 動態繪圖與數字同步顯示

Lab11-2：音樂播放器 (PWM Music Player)

本實驗利用 **PWM 輸出** 產生音樂，並搭配 **ADC** 調整音量，完成一個簡易音樂播放器：

1. 播放控制

- 按下 5：播放 〈小星星〉（自行編寫完整簡譜至 `music[]` 與 `pitch[]`）
- 按下 6：播放 〈給愛麗絲〉（參考範例 `PWM_Music`）

2. 音量調整

- 轉動可變電阻（ADC）
- 依 ADC 讀值動態調整 PWM duty cycle
- 音量大小即時變化

3. PWM 音樂原理

- 使用 PWM 產生不同頻率對應音階
- 利用 `delay` 控制音符長度
- ADC 決定音量大小

本實驗重點在於：

- PWM 頻率與音階對應
- ADC 類比輸入控制音量
- 多首樂曲切換控制

二、【遭遇的問題】：

What problems you faced during design and implementation?

- RTC 每秒更新 LCD 時，畫面容易閃爍。
- 在設定模式與運行模式切換時，時間顯示偶爾不同步。
- 秒數或分鐘超出範圍時，初期無法正確回到合法值。
- PWM 播放音樂時，若音量過小或過大，聲音失真明顯。
- ADC 讀值不穩定，導致音量忽大忽小。
- 在切換歌曲時，PWM 尚未完全停止，造成雜音。

三、【解決方法】：

How did you solve the problems?

1. 將 LCD 更新集中於固定流程，避免在主迴圈中頻繁清除畫面，減少閃爍。
2. 使用旗標（flag）區分 **Run Mode / Set Mode**，確保 RTC 在設定模式下完全暫停。
3. 在時間調整後加入範圍檢查與同餘運算，使分鐘限制在 0~11、秒數限制在 0~59。
4. 透過 PWM duty cycle 限制最大與最小音量，避免失真。
5. 對 ADC 讀值進行簡單平滑處理（平均或限制變化量），使音量變化較穩定。
6. 在切換歌曲前先停止 PWM 輸出，再重新設定頻率，避免殘留聲音。

含實驗結報的內容修改與潤飾都有使用 ChatGPT 協助。

Was there any problem that you were unable to solve? Why was it unsolvable?

- LCD 為單緩衝顯示，時鐘指針移動時仍有極輕微閃爍。
- PWM 音質受限於硬體與簡化電路，無法達到平滑音色。
- ADC 雜訊在環境干擾較大時仍會影響音量穩定性。
- 音樂播放器僅能單聲道播放，無法支援和弦或多音軌。

Lab 11-1:

[illegible]

```

47 // Wrapper for LCD library mismatch
48 void print_lcd(int line, char *str) {
49     print_Line(line, str);
50 }
51
52 // --- INTERRUPT HANDLERS ---
53
54 // 1. TIMER0 ISR (Drives the clock ticks)
55 void TMR0_IRQHandler(void)
56 {
57     TIMER_ClearIntFlag(TIMER0);
58
59     if (!g_setup_mode) {
60         g_sec++;
61         if (g_sec >= 60) g_sec = 0;
62
63         if (g_sec % 15 == 0) {
64             g_min++;
65             if (g_min >= 12) g_min = 0;
66         }
67         g_update_flag = 1;
68     }
69 }
70
71 // 2. EINT1 ISR (Mode Toggle)
72 void EINT1_IRQHandler(void)
73 {
74     GPIO_CLR_INT_FLAG(PB, BIT15);
75     g_setup_mode = !g_setup_mode;
76
77     // When setup mode starts, default to editing Minutes (or keep last selection)
78     if(g_setup_mode) {
79         PC12 = 0; // ON
80         g_edit_target = 0; // Default to Min when entering setup
81     }
82     else PC12 = 1; // OFF
83 }
84
85 // --- INITIALIZATION ---
86
87 void Init_Timer0_1Hz(void)
88 {
89     CLK_EnableModuleClock(TMR0_MODULE);
90     CLK_SetModuleClock(TMR0_MODULE, CLK_CLKSEL1_TMR0_S_HXT, 0);
91     TIMER_Open(TIMER0, TIMER_PERIODIC_MODE, 1); // 1 Hz
92     TIMER_EnableInt(TIMER0);
93     NVIC_EnableIRQ(TMR0_IRQn);
94     TIMER_Start(TIMER0);
95 }

```

```

97 void Update_Display() {
98     char text[20];
99     double angle_sec, angle_min;
100     int16_t new_sec_x, new_sec_y;
101     int16_t new_min_x, new_min_y;
102
103     // Show indicator of what we are editing in Setup Mode
104     if (g_setup_mode) {
105         if (g_edit_target == 0) sprintf(text, "%02d:%02d", g_min, g_sec);
106         else                    sprintf(text, "%02d:%02d", g_min, g_sec);
107     } else {
108         sprintf(text, "%02d:%02d ", g_min, g_sec);
109     }
110     print_lcd(0, text);
111
112     // Erase Old
113     draw_Line(old_sec_x, old_sec_y, X0, Y0, BG_COLOR, BG_COLOR);
114     draw_Line(old_min_x, old_min_y, X0, Y0, BG_COLOR, BG_COLOR);
115
116     // Calc New
117     angle_sec = (g_sec * 6 - 90) * PI / 180.0;
118     new_sec_x = X0 + (int16_t)(R_SEC * cos(angle_sec));
119     new_sec_y = Y0 + (int16_t)(R_SEC * sin(angle_sec));
120
121     angle_min = (g_min * 30 - 90) * PI / 180.0;
122     new_min_x = X0 + (int16_t)(R_MIN * cos(angle_min));
123     new_min_y = Y0 + (int16_t)(R_MIN * sin(angle_min));
124
125     // Draw New
126     draw_Line(new_min_x, new_min_y, X0, Y0, FG_COLOR, BG_COLOR);
127     draw_Line(new_sec_x, new_sec_y, X0, Y0, FG_COLOR, BG_COLOR);
128
129     old_sec_x = new_sec_x;
130     old_sec_y = new_sec_y;
131     old_min_x = new_min_x;
132     old_min_y = new_min_y;
133 }

```

```

135 int32_t main(void)
136 {
137     uint8_t key;
138     int8_t val_to_add = 0;
139
140     // 1. System Init
141     SYS_Init();
142
143     // 2. Hardware Init
144     GPIO_SetMode(PC, BIT12, GPIO_MODE_OUTPUT);
145     PC12 = 1;
146     GPIO_SetMode(PB, BIT15, GPIO_MODE_INPUT);
147     GPIO_EnableEINT1(PB, 15, GPIO_INT_FALLING);
148     NVIC_EnableIRQ(EINT1_IRQn);
149
150     // 3. Display Init
151     init_LCD();
152     clear_LCD();
153     draw_Bmp64x64(64, 0, FG_COLOR, BG_COLOR, bmp_Clock);
154
155     // 4. Timer Init
156     Init_Timer0_1Hz();
157
158     // 5. Keypad
159     OpenKeyPad();
160
161     // 6. Start
162     __enable_irq();
163
164     while(1)
165     {
166         if (g_update_flag) {
167             Update_Display();
168             g_update_flag = 0;
169         }
170
171         if (g_setup_mode) {
172             key = ScanKey();
173             if (key != 0) {
174
175                 val_to_add = 0;
176
177                 // --- 1. Mode Selectors ---
178                 if (key == 1) {
179                     g_edit_target = 0; // Edit Minutes
180                     g_update_flag = 1;
181                 }
182                 else if (key == 2) {
183                     g_edit_target = 1; // Edit Seconds
184                     g_update_flag = 1;
185                 }

```

```

187         // --- 2. Determine Value Change ---
188         else if (key == 4) val_to_add = 5;
189         else if (key == 5) val_to_add = 10;
190         else if (key == 7) val_to_add = -5;
191         else if (key == 8) val_to_add = -10;
192
193         // --- 3. Apply Change ---
194         if (val_to_add != 0) {
195             if (g_edit_target == 0) {
196                 // Modify Minutes (Range 0-11)
197                 g_min += val_to_add;
198                 // Handle Wrapping
199                 while (g_min < 0) g_min += 12;
200                 while (g_min >= 12) g_min -= 12;
201             }
202             else {
203                 // Modify Seconds (Range 0-59)
204                 g_sec += val_to_add;
205                 // Handle Wrapping
206                 while (g_sec < 0) g_sec += 60;
207                 while (g_sec >= 60) g_sec -= 60;
208             }
209             g_update_flag = 1;
210         }
211
212         CLK_SysTickDelay(200000); // Debounce
213     }
214 }
215 }
216 }

```

Lab 11-2:

```

 9  #include <stdio.h>
10  #include "NUC100Series.h"
11  #include "MCU_init.h"
12  #include "SYS_init.h"
13  #include "Note_Freq.h"
14  #include "ScanKey.h"
15
16  #define P125ms 125000
17  #define P250ms 250000
18  #define P500ms 500000
19  #define P1S    1000000
20
21  uint16_t music[72] = { // 給愛麗絲
22      E6 ,D6u,E6 ,D6u,E6 ,B5 ,D6 ,C6 ,A5 ,A5 , 0 , 0 ,
23      C5 ,E5 ,A5 ,B5 ,B5 , 0 ,C5 ,A5 ,B5 ,C6 ,C6 , 0 ,
24      E6 ,D6u,E6 ,D6u,E6 ,B5 ,D6 ,C6 ,A5 ,A5 , 0 , 0 ,
25      C5 ,E5 ,A5 ,B5 ,B5 , 0 ,E5 ,C6 ,B5 ,A5 ,A5 , 0 ,
26      B5 ,C6 ,D6 ,E6 ,E6 , 0 ,G5 ,F6 ,E6 ,D6 ,D6 , 0 ,
27      F5 ,E6 ,D6 ,C6 ,C6 , 0 ,E5 ,D6 ,C6 ,B5 ,B5 , 0 };
28
29  uint16_t test_music[58] = { // nokia 3310
30      NULL, E5,      D5,      F4u, F4u,      G4u,G4u,
31      C5u,      B4,      D4,D4,      E4,E4,
32
33      B4,      A4,      C4u,C4u,      E4,E4,
34      A4,      A4, A4
35  };

```

[illegible]

```

82 volatile uint32_t digit[4] = {0,0,0,0};
83 int cnt_5ms=0;
84
85
86 volatile uint8_t u8ADF=0;
87 int scaled_ADF=0;
88 uint32_t u32ADCvalue=0;
89 int song_index=0;
90
91 void ADC_IRQHandler(void) // 從 adc的程式抓過來的
92 {
93     uint32_t u32Flag;
94
95     // Get ADC conversion finish interrupt flag
96     u32Flag = ADC_GET_INT_FLAG(ADC, ADC_ADF_INT);
97
98     if(u32Flag & ADC_ADF_INT)
99         u8ADF = 1;
100
101     ADC_CLR_INT_FLAG(ADC, u32Flag);
102 }
103
104 void Init_ADC(void)
105 {
106     ADC_Open(ADC, ADC_INPUT_MODE, ADC_OPERATION_MODE, ADC_CHANNEL_MASK);
107     ADC_POWER_ON(ADC);
108     ADC_EnableInt(ADC, ADC_ADF_INT);
109     NVIC_EnableIRQ(ADC_IRQn);
110 }
111
112 void play_ALICE(void) { //愛麗絲
113     for (song_index=0; song_index<72; song_index++) {
114
115         ADC_START_CONV(ADC); // ADC
116         //while (u8ADF == 0);
117         u32ADCvalue = ADC_GET_CONVERSION_DATA(ADC, 7);
118         scaled_ADF = u32ADCvalue/409.6;
119
120         PWM_ConfigOutputChannel(PWM1, PWM_CH0, music[song_index], 100-scaled_ADF); // 100 為完全靜音，90剛剛好，90以下的聲音會太大
121
122
123
124
125         if (music[song_index]!=0) PWM_EnableOutput(PWM1, PWM_CH_0_MASK);
126         else PWM_DisableOutput(PWM1, PWM_CH_0_MASK);
127
128         ADC_DisableInt(ADC, ADC_ADF_INT);
129         CLK_SysTickDelay(pitch[song_index]);
130     }
131     song_index=0;
132 }

```

```

134 void play_STAR(void) { //小星星
135     for (song_index=0; song_index<144; song_index++) {
136
137         ADC_START_CONV(ADC);
138         //while (u8ADF == 0);
139         u32ADCvalue = ADC_GET_CONVERSION_DATA(ADC, 7);
140         scaled_ADF = u32ADCvalue/409.6;
141
142
143         PWM_ConfigOutputChannel(PWM1, PWM_CH0, star_music[song_index], 100-scaled_ADF); // 0=Buzzer ON
144
145         if (star_music[song_index]!=0) PWM_EnableOutput(PWM1, PWM_CH_0_MASK);
146         else PWM_DisableOutput(PWM1, PWM_CH_0_MASK);
147         CLK_SysTickDelay(star_pitch[song_index]);
148     }
149     song_index=0;
150 }
151
152 void play_TEST(void) { //nokia 3310
153     for (song_index=0; song_index<48; song_index++) {
154
155         ADC_START_CONV(ADC);
156         //while (u8ADF == 0);
157         u32ADCvalue = ADC_GET_CONVERSION_DATA(ADC, 7);
158         scaled_ADF = u32ADCvalue/409.6;
159
160
161         PWM_ConfigOutputChannel(PWM1, PWM_CH0, test_music[song_index], 80); // 0=Buzzer ON
162
163         if (test_music[song_index]!=0) PWM_EnableOutput(PWM1, PWM_CH_0_MASK);
164         else PWM_DisableOutput(PWM1, PWM_CH_0_MASK);
165         CLK_SysTickDelay(test_pitch[song_index]);
166     }
167     song_index=0;
168 }
169
170 }

```

```

172 int32_t main (void)
173 {
174     int k;
175     SYS_Init();
176     Init_ADC();
177
178     u8ADF = 0;
179     PWM_EnableOutput(PWM1, PWM_CH_0_MASK);
180     PWM_Start(PWM1, PWM_CH_0_MASK);
181
182     while(1) {
183         k = ScanKey();
184
185         /*
186         不在主程式取可變電阻的值而是在副程式迴圈裡面才取是因為
187         如果只在這裡拿可變電阻的值，那執行到副程式的時候卡在迴圈裡面播放，會取不到值，所以執行時轉可變電阻不會有反應
188
189         改成放在副程式的迴圈裡面取值就能做到邊放邊改的效果，雖然看起來有點屎山代碼 一長串 但至少能正常運作
190
191         ADC_START_CONV(ADC);
192         while (u8ADF == 0);
193         u32ADCvalue = ADC_GET_CONVERSION_DATA(ADC, 7);
194         scaled_ADF = u32ADCvalue/40.96;
195
196         printf("%d\n", u32ADCvalue);
197         printf("scaled: %d\n", scaled_ADF);
198         */
199
200         switch (k) {
201             case 5:
202                 play_ALICE();
203                 break;
204             case 6:
205                 play_STAR();
206                 break;
207             case 4:
208                 play_TEST(); //沒時間做bad apple
209                 break;
210             default:
211                 break;
212         }
213     }
214 }
215
216 }

```